

Leetcode 1: Two Sum

Code:

```
int* twoSum(int* nums, int numsSize, int target, int* returnSize) {
    *returnSize = 2;
    int* res = (int*)malloc(2 * sizeof(int));
    for (int i = 0; i < numsSize; i++) {
        for (int j = i + 1; j < numsSize; j++) {
            if (nums[i] + nums[j] == target) {
                res[0] = i; res[1] = j;
                return res;
            }
        }
    }
    return NULL;
};

}

numMap[nums[i]] = i;
}
return {};
}

};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

✓ Case 3

Input

nums = [2,7,11,15], target = 9

Output

[0,1]

Leetcode 121: Best Time to Buy and Sell Stock

Code:

```
int maxProfit(int* prices, int pricesSize) {
    int min_val = 2147483647, max_val = 0;
    for (int i = 0; i < pricesSize; i++) {
        if (prices[i] < min_val) min_val = prices[i];
        else if (prices[i] - min_val > max_val) max_val = prices[i] - min_val;
    }
    return max_val;
}

return max_prof;
};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

prices = [7,1,5,3,6,4]

Output

5

Leetcode 53: Maximum Subarray

Code:

```
int maxSubArray(int* nums, int numsSize) {  
    int curr = 0, best = -2147483648;  
    for(int i = 0; i < numsSize; i++) {  
        curr = (curr + nums[i] > nums[i]) ? curr + nums[i] : nums[i];  
        best = (curr > best) ? curr : best;  
    }  
    return best;  
}  
  
return max_sum;  
}  
};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

✓ Case 3

Input

nums = [-2,1,-3,4,-1,2,1,-5,4]

Output

6

Leetcode 88: Merge Sorted Array

Code:

```
void merge(int* nums1, int nums1Size, int m, int* nums2, int nums2Size, int n) {
    int r1 = m - 1, r2 = n - 1, w = m + n - 1;
    while (r2 >= 0) {
        if (r1 >= 0 && nums1[r1] > nums2[r2]) nums1[w--] = nums1[r1--];
        else nums1[w--] = nums2[r2--];
    }
} else {
    nums1[p--] = nums2[p2--];
}
while (p2 >= 0) {
    nums1[p--] = nums2[p2--];
}
};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3

Output

[1,2,2,3,5,6]

Leetcode 26: Remove Duplicates from Sorted Array

Code:

```
int removeDuplicates(int* nums, int numsSize) {
    if (numsSize == 0) return 0;
    int k = 1;
    for (int i = 1; i < numsSize; i++) {
        if (nums[i] != nums[i - 1]) nums[k++] = nums[i];
    }
    return k;
}
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

nums = [1,1,2]

Output

2, nums = [1,2,_]

Leetcode 189: Rotate Array

Code:

```
void reverse_arr(int* nums, int left, int right) {
    while (left < right) {
        int temp = nums[left]; nums[left++] = nums[right]; nums[right--] = temp;
    }
}

void rotate(int* nums, int numsSize, int k) {
    k %= numsSize;
    reverse_arr(nums, 0, numsSize - 1);
    reverse_arr(nums, 0, k - 1);
    reverse_arr(nums, k, numsSize - 1);
}

};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

nums = [1,2,3,4,5,6,7], k = 3

Output

[5,6,7,1,2,3,4]

Leetcode 238: Product of Array Except Self

Code:

```
int* productExceptSelf(int* nums, int numsSize, int* returnSize) {
    int* ans = (int*)malloc(numsSize * sizeof(int));
    *returnSize = numsSize;
    ans[0] = 1;
    for (int i = 1; i < numsSize; i++) ans[i] = ans[i - 1] * nums[i - 1];
    int r = 1;
    for (int i = numsSize - 1; i >= 0; i--) {
        ans[i] = ans[i] * r; r *= nums[i];
    }
    return ans;
}

for (int i = n - 1; i >= 0; i--) {
    res[i] *= right;
    right *= nums[i];
}
return res;
};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

nums = [1,2,3,4]

Output

[24,12,8,6]

Leetcode 169: Majority Element

Code:

```
int majorityElement(int* nums, int numsSize) {
    int cnt = 0, cand = 0;
    for (int i = 0; i < numsSize; i++) {
        if (cnt == 0) cand = nums[i];
        cnt += (nums[i] == cand) ? 1 : -1;
    }
    return cand;
}

int majorityElement2(int* nums, int numsSize) {
    int count = 0, candidate = 0;
    for (int i = 0; i < numsSize; i++) {
        count += (num == candidate) ? 1 : -1;
    }
    return candidate;
}

};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

nums = [3,2,3]

Output

3

Leetcode 283: Move Zeroes

Code:

```
void moveZeroes(int* nums, int numsSize) {  
    int last = 0;  
    for (int i = 0; i < numsSize; i++) {  
        if (nums[i] != 0) {  
            int t = nums[last]; nums[last++] = nums[i]; nums[i] = t;  
        }  
    }  
}
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

nums = [0,1,0,3,12]

Output

[1,3,12,0,0]

Leetcode 217: Contains Duplicate

Code:

```
int cmp(const void *a, const void *b) { return (*(int*)a - *(int*)b); }
bool containsDuplicate(int* nums, int numsSize) {
    qsort(nums, numsSize, sizeof(int), cmp);
    for (int i = 1; i < numsSize; i++) {
        if (nums[i] == nums[i - 1]) return true;
    }
    return false;
}

return false;
};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

✓ Case 3

Input

nums = [1,2,3,1]

Output

true

Leetcode 268: Missing Number

Code:

```
int missingNumber(int* nums, int numsSize) {  
    int total = numsSize * (numsSize + 1) / 2;  
    for (int i = 0; i < numsSize; i++) total -= nums[i];  
    return total;  
}
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

✓ Case 3

Input

nums = [3,0,1]

Output

2

Leetcode 448: Find All Numbers Disappeared in an Array

Code:

```
int* findDisappearedNumbers(int* nums, int numsSize, int* returnSize) {
    for (int i = 0; i < numsSize; i++) {
        int val = abs(nums[i]) - 1;
        if (nums[val] > 0) nums[val] = -nums[val];
    }
    int* res = (int*)malloc(numsSize * sizeof(int));
    int count = 0;
    for (int i = 0; i < numsSize; i++) {
        if (nums[i] > 0) res[count++] = i + 1;
    }
    *returnSize = count;
    return res;
}

vector<int> res;
for (int i = 0; i < nums.size(); i++) {
    if (nums[i] > 0) {
        res.push_back(i + 1);
    }
}
return res;
};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

nums = [4,3,2,7,8,2,3,1]

Output

[5,6]

Leetcode 75: Sort Colors

Code:

```
void sortColors(int* nums, int numsSize) {
    int l = 0, curr = 0, r = numsSize - 1;
    while (curr <= r) {
        if (nums[curr] == 0) {
            int t = nums[l]; nums[l++] = nums[curr]; nums[curr++] = t;
        } else if (nums[curr] == 2) {
            int t = nums[curr]; nums[curr] = nums[r]; nums[r--] = t;
        } else { curr++; }
    }
    } else if (nums[mid] == 1) {
        mid++;
    } else {
        swap(nums[mid], nums[high--]);
    }
    }
}
};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

nums = [2,0,2,1,1,0]

Output

[0,0,1,1,2,2]

Leetcode 31: Next Permutation

Code:

```
void nextPermutation(int* nums, int numsSize) {
    int i = numsSize - 2;
    while (i >= 0 && nums[i] >= nums[i + 1]) i--;
    if (i >= 0) {
        int j = numsSize - 1;
        while (nums[j] <= nums[i]) j--;
        int t = nums[i]; nums[i] = nums[j]; nums[j] = t;
    }
    for (int l = i + 1, r = numsSize - 1; l < r; l++, r--) {
        int t = nums[l]; nums[l] = nums[r]; nums[r] = t;
    }
}

if (i >= 0) {
    int j = nums.size() - 1;
    while (j >= 0 && nums[j] <= nums[i]) {
        j--;
    }
    swap(nums[i], nums[j]);
}
reverse(nums.begin() + i + 1, nums.end());
};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

✓ Case 3

Input

nums = [1,2,3]

Output

[1,3,2]

Leetcode 42: Trapping Rain Water

Code:

```
int trap(int* height, int heightSize) {
    int l = 0, r = heightSize - 1;
    int maxL = 0, maxR = 0, res = 0;
    while (l <= r) {
        if (maxL < maxR) {
            height[l] > maxL ? (maxL = height[l]) : (res += maxL - height[l]); l++;
        } else {
            height[r] > maxR ? (maxR = height[r]) : (res += maxR - height[r]); r--;
        }
    }
    return res;
} else {
    height[right] >= right_max ? (right_max = height[right]) : water += (right_max - height[right]);
    right--;
}
return water;
};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output

6

Leetcode 11: Container With Most Water

Code:

```
int maxArea(int* height, int heightSize) {
    int l = 0, r = heightSize - 1, maxA = 0;
    while (l < r) {
        int h = height[l] < height[r] ? height[l] : height[r];
        int area = h * (r - l);
        if (area > maxA) maxA = area;
        if (height[l] < height[r]) l++;
        else r--;
    }
    return maxA;
} else {
    right--;
}
return max_water;
};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

height = [1,8,6,2,5,4,8,3,7]

Output

49

Leetcode 134: Gas Station

Code:

```
int canCompleteCircuit(int* gas, int gasSize, int* cost, int costSize) {
    int tGas = 0, tCost = 0, cGas = 0, start = 0;
    for (int i = 0; i < gasSize; i++) {
        tGas += gas[i]; tCost += cost[i];
        cGas += gas[i] - cost[i];
        if (cGas < 0) { start = i + 1; cGas = 0; }
    }
    return tGas >= tCost ? start : -1;
}

return total_gas >= total_cost ? start_station : -1;
};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

gas = [1,2,3,4,5], cost = [3,4,5,1,2]

Output

3

Leetcode 55: Jump Game

Code:

```
bool canJump(int* nums, int numsSize) {  
    int max_r = 0;  
    for (int i = 0; i < numsSize; i++) {  
        if (i > max_r) return false;  
        if (i + nums[i] > max_r) max_r = i + nums[i];  
    }  
    return true;  
}  
  
return true;  
};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

nums = [2,3,1,1,4]

Output

true

Leetcode 45: Jump Game II

Code:

```
int jump(int* nums, int numsSize) {  
    int jumps = 0, curr_end = 0, max_reach = 0;  
    for (int i = 0; i < numsSize - 1; i++) {  
        if (i + nums[i] > max_reach) max_reach = i + nums[i];  
        if (i == curr_end) {  
            jumps++; curr_end = max_reach;  
        }  
    }  
    return jumps;  
}  
  
return jumps;  
};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

nums = [2,3,1,1,4]

Output

2

Leetcode 135: Candy

Code:

```
int candy(int* ratings, int ratingsSize) {
    int* c = (int*)malloc(ratingsSize * sizeof(int));
    for (int i = 0; i < ratingsSize; i++) c[i] = 1;
    for (int i = 1; i < ratingsSize; i++) {
        if (ratings[i] > ratings[i - 1]) c[i] = c[i - 1] + 1;
    }
    int total = c[ratingsSize - 1];
    for (int i = ratingsSize - 2; i >= 0; i--) {
        if (ratings[i] > ratings[i + 1] && c[i] <= c[i + 1]) c[i] = c[i + 1] + 1;
        total += c[i];
    }
    free(c);
    return total;
}

}

for (int i = n - 2; i >= 0; i--) {
    if (ratings[i] > ratings[i + 1]) {
        candies[i] = max(candies[i], candies[i + 1] + 1);
    }
}

int total = 0;
for (int c : candies) total += c;
return total;
}

};
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

ratings = [1,0,2]

Output

5